

# Agente RAG: Asistente Inteligente de Conocimiento

Vicente Rivas Monferrer & Juan M. Alberola

jalberola@dsic.upv.es

Grado en Tecnologías Interactivas

## Recursos

### Código base:

- <https://github.com/vjrivmon/agente-rag-gti>

### Servidores LLM:

- **Ollama local:** instalado en vuestro portátil. Es **obligatorio** que el sistema funcione con Ollama en vuestro local (un sistema que sólo funcione con PoliGPT no llega al 5).
- **PoliGPT UPV:** <https://api.poligpt.upv.es>. **Requiere VPN de la UPV** desde fuera del campus.

### Documentación recomendada:

#### ■ Frameworks que vais a usar en la práctica:

- **ChromaDB:** vector store por defecto, persistente en disco.
- **LangChain:** del paquete oficial usaréis `langchain-text-splitters` (`RecursiveCharacterTextSplitter`) para chunking.
- **Sentence-Transformers:** alternativa local a los embeddings de Ollama si no queréis depender de la red.

#### ■ Recuperación de información (retrieval):

- **LangChain — Retrievers:** panorama de tipos de retriever (semántico, BM25, MMR, híbrido) y cuándo usar cada uno.
- **ChromaDB — Querying Collections:** API de búsqueda top-*k* con filtros y métricas de similitud.
- **Sentence-Transformers — Semantic Search:** receta completa de búsqueda semántica con embeddings, incluyendo *re-ranking*.
- **rank-bm25:** implementación ligera de BM25 en Python; útil para añadir un canal léxico que complemente al semántico.
- **Lewis et al. (2020) — Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks:** el paper que introdujo el término RAG; lectura corta y formativa.

#### ■ Evaluación:

- **RAGAs**: framework de métricas específicas para sistemas RAG (banda 8).
- **Extras opcionales**:
  - **AWS Rekognition**.
  - **Hugging Face**.

**Agent-CLIs e IDEs (no obligatorios; ver Anexo C):**

- Open source: **OpenCode**, **Aider**, **Zed**.
- De pago: **Claude Code**, **Cursor**, **Gravity**, **Zen**.

## Introducción

Los sistemas de **Retrieval-Augmented Generation (RAG)** representan uno de los avances más relevantes en la aplicación práctica de la Inteligencia Artificial. Un agente RAG combina la capacidad de recuperar información relevante de una base de conocimientos con la generación de respuestas en lenguaje natural mediante un modelo de lenguaje (LLM). A diferencia de un LLM estándar que utiliza únicamente su conocimiento interno (propenso a generar información falsa o alucinaciones), un sistema RAG fundamenta sus respuestas en documentos reales, ofreciendo respuestas precisas y verificables.

En esta práctica, desarrollaréis **en parejas o grupos de tres personas** un agente RAG capaz de responder preguntas sobre la asociación **DNI (Damos Nuestra Ilusión) Valencia**. Esta asociación gestiona múltiples proyectos sociales: desayunos solidarios para personas sin hogar, visitas a residencias de ancianos (RESIS), refuerzo escolar para niños en riesgo de exclusión (COLES), entre otros.

El objetivo es construir un sistema que, dada una pregunta del usuario, recupere la información relevante de los documentos proporcionados y genere una respuesta precisa, **citando las fuentes utilizadas**. El sistema **no debe inventar información** que no esté en la base de conocimientos.

## Qué vais a construir

Un **agente** es un sistema que **PERCIBE** una pregunta, **DECIDE** qué chunks recuperar y cómo construir el prompt, y **ACTÚA** devolviendo una respuesta fundamentada con sus fuentes y, cuando proceda, rechazando preguntas que no puede responder.

El agente debe de responder sobre la asociación **DNI**, una asociación juvenil de voluntariado en Valencia con más de 400 voluntarios en activo. El corpus cubre proyectos como desayunos solidarios, refuerzo escolar y charlas en residencias de mayores.

Lo que distingue un agente de un LLM a secas

Un **agente** cumple los tres verbos: **percibir**, **decidir**, **actuar**. Si vuestro sistema solamente le pasa una pregunta a un LLM y devuelve la respuesta tal cual, no es un agente: es una llamada a una API. Vuestro agente debe orquestrar al menos: recuperación de contexto, construcción de prompt, llamada al LLM y formateo de salida con fuentes citadas.

## ¿Qué es un sistema RAG?

Un sistema **RAG (Retrieval-Augmented Generation)** es una arquitectura que mejora las respuestas de un modelo de lenguaje proporcionándole **contexto relevante** extraído de una base documental propia. A diferencia de un LLM que responde sólo desde su conocimiento interno (propenso a alucinar y a cometer errores no deseados), un RAG fundamenta cada respuesta en documentos reales y permite citar la fuente exacta.

El proceso se divide en dos fases (Figura 1):

### 1. Fase de indexación (offline, una única vez):

- Cargar la base de conocimientos (documentos de texto en formato .txt)
- Dividirlos en fragmentos manejables (*chunks*).
- Convertir cada fragmento en un vector numérico (*embedding*).
- Almacenar los vectores en un *vector store*.

### 2. Fase de consulta (online, en cada pregunta del usuario):

- Convertir la pregunta del usuario en un vector.
- Buscar los fragmentos más similares en el *vector store*.
- Construir un *prompt* con la pregunta y el contexto recuperado.
- Enviar el *prompt* al LLM para que genere la respuesta.
- Devolver la respuesta junto con el contexto consultado en las fuentes.

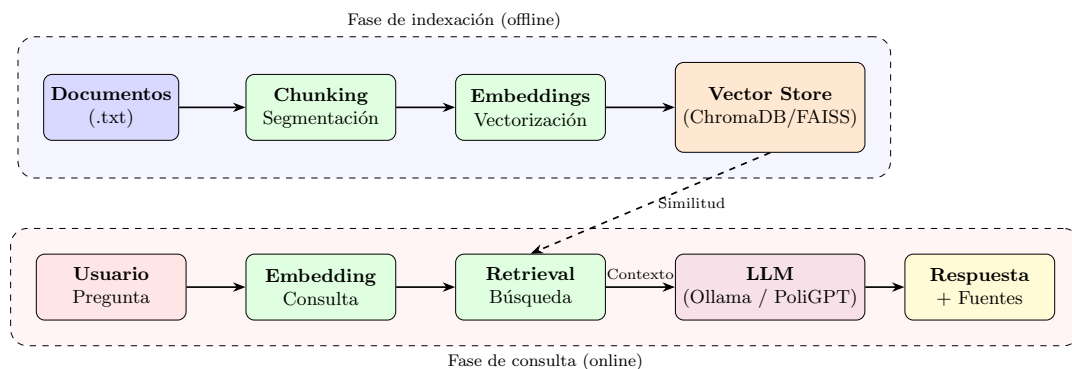


Figura 1: Arquitectura de un sistema RAG. La fase de indexación se ejecuta una vez (al arrancar el agente o cuando cambia el corpus). La fase de consulta se ejecuta en cada pregunta del usuario.

## Pipeline de procesamiento de una consulta

La Figura 2 desglosa paso a paso lo que ocurre dentro del agente cuando un usuario formula una pregunta. Vuestro `consultar.py` debe implementar exactamente este flujo (con las variantes de chunking,  $k$ , prompt y modelo que justifiquéis en el informe).

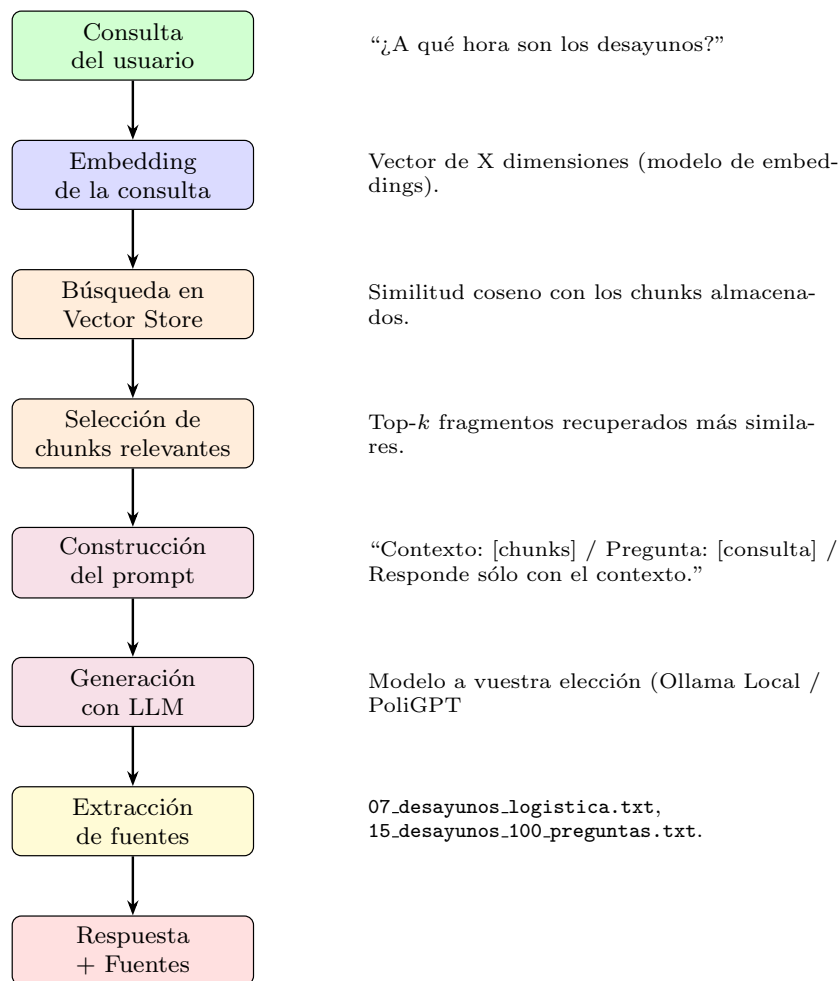


Figura 2: Pipeline de procesamiento de una consulta. Cada paso se corresponde con un componente identificable en `consultar.py` (single-agent) o con una clase del dominio (hexagonal).

## Base de conocimiento DNI

Se proporcionan **16 ficheros de texto** con toda la información oficial de la asociación DNI. Los temas cubiertos son:

- **Información general:** qué es DNI, filosofía, valores y preguntas básicas.
- **Proyectos:** Desayunos Solidarios, Residencias de Mayores, Refuerzo Escolar.
- **Logística:** horarios, ubicaciones, cómo participar, contacto.
- **Preguntas frecuentes:** 100 preguntas sobre Desayunos, 49 sobre Residencias.
- **Impacto social:** testimonios y datos.

Algunos ficheros usan formato **Q:/A:** (pregunta/respuesta), donde cada par forma una unidad semántica natural, mientras que otros son narrativos. El sistema debe ser capaz de manejar ambos

formatos. En este sentido, es importante detectar y respetar esa estructura mejora la calidad del retrieval, ya que si vuestro chunker corta entre la pregunta y su respuesta, perdéis contexto importante.

#### Aviso importante sobre contradicciones

El corpus contiene contradicciones reales entre documentos (por ejemplo, un fichero dice que los desayunos son a las 8:00 y otro a las 7:00). **Esa contradicción forma parte del corpus y NO debe corregirse.** Vuestro agente debe ser capaz de detectarla y, si procede, presentar las dos versiones citando ambas fuentes. Inventar una respuesta única que oculte la contradicción es alucinación y penaliza.

El corpus pesa ~92 KB en total. Es deliberadamente pequeño, lo que permite que quepa entero en memoria, se indexe en segundos, y permita iterar rápido sobre la calidad de las respuestas sin coste computacional.

**Importante:** el sistema solo debe usar la información de estos 16 ficheros. No debe inventar datos, fechas, horarios ni contactos que no aparezcan literalmente en los documentos. Si se detecta un cambio en la información de los 16 ficheros, se penalizará acorde al cambio realizado.

## Teoría mínima de Agentes LLM

En esta práctica vais a construir un **agente LLM**: un sistema en el que un modelo de lenguaje actúa como *cerebro* de la toma de decisiones. La diferencia respecto a una llamada directa al LLM es justo esa: en vez de *pregunta entra, respuesta sale*, hay un **loop de decisiones intermedias** (qué información buscar, cómo construir el prompt, cuándo rechazar la pregunta) que el agente orquesta antes de responder. Un agente LLM se compone, como mínimo, de cinco piezas:

- **Modelo de lenguaje:** el componente cognitivo que toma decisiones y genera la respuesta final. En vuestro caso, el LLM accesible vía Ollama o PoliGPT.
- **Percepción:** cómo entra la información al agente (la pregunta del usuario y, eventualmente, contexto adicional como un identificador de conversación o, en los extras, una imagen o un audio).
- **Memoria:** qué retiene entre interacciones. Distinguid memoria de contexto (lo que cabe en el prompt) y memoria a largo plazo (vector store, base de datos, ficheros).
- **Herramientas:** funciones invocables por el agente (recuperar chunks, llamar a una API externa, leer un fichero...). En vuestra práctica, la herramienta principal es el retriever sobre el corpus DNI.
- **Planificación:** la lógica que decide qué hacer y en qué orden. En la versión más simple es una secuencia fija (recuperar → construir prompt → llamar al LLM → formatear salida); en versiones más sofisticadas el LLM mismo decide qué herramienta usar y cuándo.

#### Decisión que tenéis que tomar antes de empezar (o no habiendo avanzado mucho)

Si aspiráis al 10, **empezad desde un inicio con la arquitectura hexagonal**. Refactorizar de una arquitectura single-agent a una hexagonal cuando ya tenéis un 8 asegurado son varios días de trabajo y no merece la pena. Si os conformáis con un 9, podéis empezar con un single-agent y sumar los extras que hay disponibles.  
Para aprender y saber más de la arquitectura hexagonal: ver `manual_tecnico_dni.pdf`.

## El contrato de interfaz (obligatorio)

Para desarrollar el código, debéis seguir ciertas indicaciones definidas en el siguiente contrato de interfaz. Sin respetar este contrato, el corrector no puede evaluar vuestro sistema y la nota será penalizada. Este contrato **aplica tanto a single-agent como a hexagonal**: lo que cambia es cómo lo organizáis internamente. En single-agent, todo el flujo cabe en `consultar.py` (o detrás del endpoint). En hexagonal, `consultar.py` (o el endpoint HTTP) actúa como *adapter* de entrada que invoca el dominio. Por tanto, debéis elegir **una** de las siguientes dos opciones:

### Opción A — Módulo Python

Vuestro sistema expone una única función Python en la raíz del repositorio que el corrector importará y llamará. Es la opción más sencilla y la recomendada si no tenéis ya un servidor HTTP montado por otros motivos. Implementad en `consultar.py` la siguiente función:

```
def consultar(pregunta: str,
              conversation_id: str | None = None) -> dict:
    return {
        "respuesta": str,          # texto de la respuesta
        "fuentes": list[str],      # ["07_desayunos_logistica.txt", ...]
        "chunks": list[str] | None, # opcional; obligatorio para banda 7
        "metricas": dict | None,   # tokens y latencia (banda 7)
        "trazas": list[dict] | None, # opcional
    }
```

El corrector hará `from consultar import consultar` y enviará una pregunta por llamada, comprobando la forma del diccionario devuelto.

### Opción B — Endpoint HTTP

Vuestro sistema expone un servidor HTTP que el corrector consulta vía `POST`. Útil si ya tenéis frontend o si preferís separar el agente como servicio independiente. Servid `POST /query` con el mismo JSON que la opción A. El servidor debe estar corriendo durante toda la corrección (recomendado: `uvicorn` con FastAPI o equivalente). Indicad puerto y URL en `features.json` (ver Sección ); el corrector hará la petición a esa URL exacta.

## Bandas de notas

En función del trabajo realizado, se definen varias bandas de notas que permiten ir desde un 5 hasta un 10. A continuación, se detallan los requisitos para llegar a cada una de estas bandas de notas.

### Banda 5 — Mínimo para aprobar

- Pipeline RAG completo: carga `.txt` → chunking → embeddings → vector store (ChromaDB o equivalente) → retrieval semántico → LLM → respuesta.
- Prompt anti-alucinación: ante una pregunta cuya respuesta no esté en el corpus, el sistema responde literalmente *“No tengo esa información en mis fuentes”* (o equivalente claro). **No inventa.**

- El sistema funciona con **Ollama instalado en la máquina del alumno**. Un sistema que solo funcione con PoliGPT no llega al 5.
- Informe breve donde se describa la arquitectura, pipeline, decisiones tomadas y dificultades encontradas. Debe notarse que es algo personal y no un copy-paste de Chatgpt.
- `AI_USAGE.md` honesto y rellenado
- `README` que explique vuestra solución.
- `GRUPO.md` con los integrantes de cada equipo y la función realizada dentro del mismo

## Banda 6 — Cita del archivo fuente

Todo lo de banda 5 **más**:

- Cada respuesta cita el **archivo fuente** del que procede el chunk usado, como por ejemplo: fuentes: `["07_desayunos_logistica.txt", "09_como_participar.txt"]`.
- Las fuentes citadas **contienen realmente** la información de la respuesta. Si un archivo no aparece en el contexto recuperado, no puede citarse.
- El informe documenta el chunking aplicado (tamaño, overlap, justificación).

## Banda 7 — Benchmark con 4 modelos

Todo lo de banda 6 **más**:

- **Benchmark documentado** con **4 modelos** sobre el mismo set de preguntas. Habréis de probar:
  - **2 modelos en PoliGPT** (`api.poligpt.upv.es`, requiere VPN UPV fuera del Campus). El catálogo cambia con el tiempo, así que comprobad qué hay disponible antes de fijar los dos modelos: con vuestra clave podéis hacer `GET https://api.poligpt.upv.es/v1/models` y os devuelve la lista actual.
  - **2 modelos locales** vía Ollama.
- Cada respuesta del benchmark debe registrar como mínimo: tokens de entrada, tokens de salida, latencia (s), tokens/s, calidad subjetiva (acierto/fallo en cada pregunta).
- Tabla/Gráfico en el informe con los resultados obtenidos y una interpretación subjetiva de los mismos.
- Una breve discusión de los hallazgos más importantes después de ejecutar el benchmark.
- Todo esto se incluiría en el informe.

**Cómo organizar el benchmark.** Un benchmark, en esencia, ejecuta el mismo set de preguntas contra distintos modelos y registra las mismas métricas para poder compararlas. Para esta práctica os recomendamos esta receta:

1. **Set fijo de preguntas** (10–20). Mezclad factuales directas, síntesis multi-doc y al menos una pregunta fuera de ámbito. Guardadlas en `benchmark/preguntas.json`.
2. **Un único pipeline parametrizado por modelo**. Sólo cambia el modelo; el resto (chunking, retrieval, prompt) se mantiene constante. Si no, no estaríais comparando modelos sino sistemas distintos.

3. Un **script reproducible** (`benchmark.py` o similar) que itera sobre cada par (`modelo`, `pregunta`) y guarda `benchmark/benchmark.json` con respuesta y métricas crudas.
4. Un **postproceso** que produce `benchmark/benchmark.md` con la tabla legible (tokens, latencia, acierto subjetivo) y un párrafo de interpretación.

En el Colab tenéis cómo se obtienen `prompt_eval_count`, `eval_count` y `eval_duration` de la respuesta de Ollama, que son los datos fuente para tokens y latencia. Para PoliGPT la información equivalente está en `response.usage`.

## Banda 8 — RAGAs y métricas propias

Todo lo de banda 7 **más**:

- Evaluación con **RAGAs**, framework específico para evaluar sistemas RAG ([docs.ragas.io](https://docs.ragas.io)). Debéis incluir las cuatro métricas siguientes:
  - **faithfulness**: grado de fidelidad de la respuesta al contexto recuperado. Detecta cuándo el agente inventa información que no estaba en los chunks.
  - **answer\_relevancy**: cuánto se ciñe la respuesta a la pregunta planteada (penaliza divagaciones o respuestas tangenciales).
  - **context\_precision**: qué proporción de los chunks recuperados aporta información realmente útil para la respuesta.
  - **context\_recall**: qué proporción de la información necesaria para responder está presente en los chunks recuperados.

Para usarlas, RAGAs necesita por cada pregunta: la pregunta, los chunks recuperados (contexto), la respuesta del agente y, para algunas métricas, una respuesta de referencia (*ground truth*). La respuesta de referencia podéis redactarla a mano para vuestro set de preguntas.

- **2 métricas propias** definidas y justificadas.
- Resultados RAGAs y métricas propias **integrados en la tabla/gráfica** del benchmark realizado. La elección del modelo elegido ha de defenderse con estos datos (debe de ser acorde en el informe).
- Todo esto se incluiría en el informe.

## Banda 10 — Arquitectura hexagonal

Todo lo de banda 8 **más**:

- **Arquitectura hexagonal completa**:
  - Dominio puro (entidades + lógica de orquestación) sin imports de infraestructura.
  - **Ports**: como mínimo `ILLMClient`, `IEmbeddingsClient`, `IVectorStore`, `IRetriever`.
  - **Adapters**: como mínimo **dos implementaciones de cada port** (ej. `OllamaLLMAdapter` y `PoliGPTLLMAdapter`; `ChromaAdapter` y `FAISSAdapter`). Cambiar de adapter = 1 línea de configuración.
  - Tests del dominio que mockean los ports y **no requieren red**.



- README profesional con diagrama de capas e instrucciones para añadir un adapter nuevo.
- Análisis crítico: limitaciones identificadas + propuesta de mejora futura.
- Todo esto se incluiría en el informe.

## Presentación Oral

Una vez pase la fecha de entrega, se realizará una presentación oral, donde cada equipo deberá seguir este procedimiento:

1. Se os formulará **una pregunta** sobre DNI.
2. Después, se observa la respuesta del agente, teniendo en cuenta lo siguiente:
  - **Acierto** (cita la fuente correcta, no inventa, formato razonable).
  - **Aproximación con erratas menores** (información correcta pero sin fuente, o cita parcial): banda 5 condicionada a la corrección del resto del informe.
  - **Alucinación** (inventa información que no está en el corpus): nota condicionada a la revisión profunda de la entrega y del código.
3. Si el grupo aspira a banda 8 o 10 o declara extras, se puede formular una segunda pregunta orientada a ello .

## Extras (suman sobre la nota final)

A parte de las bandas de notas que se han comentado anteriormente, también se pueden sumar puntos con 3 posibles extras, dejando una nota máxima de 9 en el caso de optar por una arquitectura single-agent:

| Extra           | Cuándo aplica                                                                                                                      | Suma |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------|------|
| Frontend        | Streamlit / Gradio / Next.js / FastAPI+HTML funcional, expone el agente, no es solo cosmético.                                     | +1,5 |
| AWS Rekognition | Subir una foto (cartel, horario impreso, post de IG...) y el agente verifica si el contenido coincide con el corpus oficial.       | +1,5 |
| Hugging Face    | TTS, STT, NER, clasificación, etc., con un modelo HF integrado en el flujo y útil. No vale “hago una llamada y tiro la respuesta”. | +1,5 |

## Material a entregar

Repositorio y ZIP del código fuente de la práctica. Una única entrega por grupo en Poliformat, siguiendo la siguiente estructura.

## Código y configuración

- `consultar.py` (o indicar endpoint HTTP en `features.json`).
- `requirements.txt` con las versiones de los paquetes adecuada.
- `.env.example` (sin valores reales, solo de ejemplo).

- `features.json` siguiendo la plantilla de la Sección .
- `base_conocimiento/` con los 16 `.txt` sin modificar.
- Si hexagonal: `domain/`, `ports/`, `adapters/` con la estructura convencional.

## Documentos obligatorios

- `README.md` con explicaciones adecuadas de lo que consiste vuestro código.
- `informe.pdf` ( $\geq 2$  páginas) con:
  - Arquitectura elegida **justificando vuestra elección**.
  - Decisiones tomadas a lo largo del desarrollo.
  - Flujo detallado de una consulta (qué hace cada componente y en qué orden).
  - Resultados sobre las preguntas de prueba realizadas.
  - Análisis crítico, limitaciones y propuestas de mejora.
  - Dificultades encontradas y cómo las resolvisteis.
- `AI_USAGE.md` (honesto y específico). Por cada herramienta IA usada (Claude Code, OpenCode, Cursor...) indicar: qué tareas hizo, qué ficheros modificó, qué partes del código son fruto de IA sin vuestra posterior revisión.
- `benchmark/` (banda 7+):
  - `benchmark.json` con resultados estructurados.
  - `benchmark.md` con tabla legible.
  - Scripts que generan ambos.
- `evaluacion/` (banda 8+):
  - `ragas_results.json` con las 4 métricas RAGAs.
  - `metricas_propias.md` con definición + valores de las 2 métricas que habéis inventado.
- `GRUPO.md` listando: nombres y apellidos de los integrantes de cada grupo y el rol que ha asumido cada uno.

## Fichero `features.json` — declaración obligatoria en la raíz del repositorio

Independientemente de la opción del contrato (A o B) y de la arquitectura (single-agent o hexagonal), **todos los grupos entregan `features.json` en la raíz del ZIP**. El corrector lo abre antes que cualquier otro fichero: declara qué bandas y extras habéis implementado realmente, y **sólo se evalúa lo que esté en `true`**. Marcar con `true` algo no implementado es trampa y penaliza adicionalmente. A continuación tenéis la plantilla que podéis utilizar para definir este fichero. Los valores con `|` son las alternativas posibles; en vuestro fichero final dejaréis sólo el valor escogido, no la barra:

```
{
  "_": "Marca true las capacidades que tu equipo IMPLEMENTA realmente.
        Declarar algo en true sin tenerlo es trampa y penaliza.",
  "interfaz": "opcion_a | opcion_b",
  "modulo": "consultar.py",
```

```

"endpoint_http": null,
"arquitectura": "single_agent | hexagonal",
"bandas_declaradas": {
  "banda5_pipeline_no_inventa": true | false,
  "banda6_cita_archivo": true | false,
  "banda7_benchmark_4_modelos": true | false,
  "banda8_ragas_metricas_propias": true | false,
  "banda10_hexagonal": true | false
},
"extras_declarados": {
  "frontend": true | false,
  "aws_rekognition": true | false,
  "huggingface": true | false
},
"modelos_evaluados_en_benchmark": [
  {"alias": "gemma3:4b", "servidor": "ollama_local"},
  {"alias": "gemma3:27b", "servidor": "poligpt"}
],
"ollama_local_obligatorio": false,
"verify_ssl_upv": false
}

```

#### Campos que debéis tocar:

- **interfaz**: "opcion\_a" si entregáis `consultar.py`; "opcion\_b" si servidor HTTP.
- **modulo**: nombre del fichero con la función `consultar` (por defecto `consultar.py`). Sólo se mira si `interfaz="opcion_a"`.
- **endpoint\_http**: URL completa (`http://localhost:PUERTO/query`) si `interfaz="opcion_b"`; si no, `null`.
- **arquitectura**: "single\_agent" (cap 9) o "hexagonal" (cap 10).
- **bandas\_declaradas.bandaX**: marcad `true` sólo las que cumpláis *todas* sus condiciones (Bandas 5–10).
- **extras\_declarados.X**: marcad `true` sólo si lo habéis implementado al completo y funciona en vuestro portátil.
- **modelos\_evaluados\_en\_benchmark**: lista de los **4 alias** que aparecen en `benchmark/benchmark.json` (banda 7+), con su servidor ("ollama\_local" o "poligpt"). Vacío si no hacéis benchmark.

#### Errores que invalidan la entrega automáticamente:

- `features.json` ausente o JSON mal formado.
- Fichero presente pero no es un objeto JSON (lista, número, cadena).
- `interfaz="opcion_a"` pero no existe `consultar.py` en la raíz.
- `interfaz="opcion_b"` pero `endpoint_http` es `null` o vacío.
- Banda declarada `true` sin los artefactos esperados (p. ej. `banda7_benchmark_4_modelos: true` sin `benchmark/benchmark.json`).

## Criterios de evaluación

La nota se construye en **tres pasos secuenciales**:

1. **Banda alcanzada:** Se verifican los requisitos de cada banda en orden.
2. **Ajuste dentro de la banda:** la nota final dentro de la banda depende de los criterios cualitativos de la tabla siguiente.
3. **Suma de extras y aplicación de nota máxima:** se suman los extras declarados y se aplica el máximo dentro de cada arquitectura (9 si single-agent, 10 si hexagonal). Después se aplican las penalizaciones que se comentan a continuación.

### Criterios de ajuste dentro de la banda

| Elemento                                 | Qué se valora                                                                                                                                                     |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Calidad del código</b>                | Legibilidad, separación de responsabilidades, naming, manejo de errores, ausencia de duplicación (se aplica el uso de código limpio DRY - Don't Repeat Yourself). |
| <b>Calidad del informe</b>               | Justificación de la solución aplicada, profundidad del análisis, claridad de lo que se comenta.                                                                   |
| <b>Rigor del benchmark</b><br>(banda 7+) | Reproducibilidad, datos limpios y completos, análisis textual de los resultados (no solo tabla).                                                                  |
| <b>Métricas propias</b> (banda 8+)       | Pertinencia al dominio DNI, definición precisa, justificación.                                                                                                    |
| <b>Hexagonal</b> (banda 10)              | Limpieza del dominio (sin imports externos), intercambiabilidad real de adapters demostrada, tests del dominio sin red.                                           |
| <b>AI.USAGE.md</b>                       | Especificidad por herramienta y fichero, honestidad sobre código no revisado.                                                                                     |
| <b>Historial git</b>                     | Commits frecuentes, descriptivos y pequeños.                                                                                                                      |
| <b>Presentación Oral</b>                 | Que responda correctamente sin alucinar es como mínimo banda 5. Aplica también a defensa de extras.                                                               |

**Resumen de la fórmula:**

$$\text{Nota final} = \min \left( \max_{\text{arq}}, \text{banda} + \sum \text{extras} \right) - \sum \text{penalizaciones}$$

donde  $\max_{\text{arq}} = 9$  para single-agent y 10 para hexagonal.

## Penalizaciones

### Lista de penalizaciones

- Sin AI.USAGE.md o vacío: como máximo optas al 5.
- No se siguen las especificaciones del material a entregar: entre 1 y 5 puntos menos.
- No se detalla correctamente features.json: entre 1 y 5 puntos menos.
- Si se detectan respuestas hardcodeadas en alguna parte del código: 5 puntos menos.
- El sistema sólo funciona con PoliGPT: máximo un 5.

## Anexo A — Preguntas tipo

Para que os hagáis a una idea, las preguntas serán de este estilo:

- **Factual directa:** “¿Qué es DNI?”.
- **Logística práctica:** “¿Cómo me apunto a los desayunos solidarios?”.
- **Síntesis multi-doc:** “¿En qué se diferencian RESIS y COLES?”.
- **Fuera de ámbito:** “¿Cuánto cuesta el alquiler en Valencia?”.

## Anexo B — Conexión con Ollama (local) y PoliGPT

### Ollama local

```
import requests

def llamar_ollama(prompt: str, modelo: str = "qwen2.5:3b") -> str:
    response = requests.post(
        "http://localhost:11434/api/generate",
        json={
            "model": modelo,
            "prompt": prompt,
            "stream": False,
            "options": {"temperature": 0.2, "num_predict": 500}
        },
        timeout=300,
    )
    return response.json()["response"]
```

### PoliGPT

```
import os
from openai import OpenAI

client = OpenAI(
    base_url="https://api.poligpt.upv.es/v1",
    api_key=os.environ["POLIGPT_API_KEY"],
)

resp = client.chat.completions.create(
    model="poligpt",
    messages=[{"role": "user", "content": "..."}],
    temperature=0.2,
)
print(resp.choices[0].message.content)
```

## Anexo C — Buenas prácticas con IDEs agénticas

Podéis usar cualquier agent-CLI o IDE con LLM integrado para construir vuestra práctica. Ninguna es obligatoria.

#### Tendencias 2026

**Open source / gratis:** OpenCode, Aider, Zed.

**De pago:** Claude Code, Cursor, Gravity, Zen.

Lo que uséis no afecta a la nota; lo que afecta es **cómo** lo uséis.

#### Cuatro reglas, cualquier herramienta:

1. **Entended lo que se está haciendo en todo momento.** Si el agente produce código que no sabríais explicar mejor que no lo entreguéis.
2. **Dejad huella en git.** Commits frecuentes, descriptivos, pequeños. Un repo con dos commits (“initial” y “final”) de 2000 líneas cada uno es una mala señal y significa no haber llevado a cabo unas buenas prácticas con git.
3. **Pedid que os genere un plan antes de codificar directamente.** Casi todos los CLIs permiten “modo plan”. De esta manera os aseguráis que realmente lo que le habéis dicho de hacer es lo que ha entendido y ha planificado. Cualquier error de comprensión que haya podido tener lo podréis detectar rápidamente de esta manera, revisadlo con calma.
4. **Declarad todo en AI\_USAGE.md.** No ocultéis el uso. Sé honesto y específico por herramienta y fichero.

**Sugerencia operativa:** tened un fichero de instrucciones para vuestro CLI (**AGENTS.md** para OpenCode, **CLAUDE.md** para Claude Code, **.cursorrules** para Cursor) con idioma de comentarios, estilo, estructura de carpetas. El CLI lo respetará.